AI VIET NAM | All-in-One Course 2026

Hands-on: Kiến trúc Apache Kafka Partition, group, lag và replication

Hướng dẫn thực hành — Kỹ thuật dữ liệu

Tạo topic nhiều partition, quan sát phân phối message, consumer group, lag, replay và replication

Bài đọc nền tảng: Kiến trúc Apache Kafka.

Mục tiêu buổi lab

Sau buổi lab, học viên tự khởi động Kafka bằng Docker, tạo topic có 3 partition và gửi sự kiện có key. Học viên dùng bằng chứng từ công cụ để giải thích bốn điều: cùng key nằm cùng partition, consumer trong một group chia nhau partition, lag đo theo offset, và replay là đặt lại committed offset. Phần mở rộng dùng cluster 3 broker để quan sát leader, ISR và `acks`.

Môi trường: Windows, PowerShell và Docker Desktop chạy Linux containers. Các lệnh Kafka chạy trong container, nên không cần cài Java hoặc Kafka trực tiếp trên Windows.

Lưu ý

Trạng thái: toàn bộ lab, kể cả phần mở rộng, đã được chạy end-to-end ngày 10/11/2026 trên Windows 11, Windows PowerShell 5.1, Docker Engine 29.9.2 và image `apache/kafka:4.1.2`. Các khối "Kết quả quan sát" là output của lượt chạy đó, đã thay tab bằng khoảng trắng và rút gọn các cột/ID dài. Topic ID, consumer ID và thời gian sẽ khác trên máy của bạn; số record theo partition được giải thích ở từng bước.

Mục lục

1. **Thiết kế buổi lab** 2
2. **Mô hình lab và khởi động Kafka** 3
3. **Tạo topic với ba partition** 4
4. **Tạo và gửi 60 record có key** 5

5. Đọc kèm partition, offset và key **6**
6. Hai consumer chia việc trong cùng group **7**
7. Đọc độc lập bằng một group khác **9**
8. Bốn consumer trên ba partition **10**
9. Consumer lag, reset offset và replay **10**
10. Mở rộng: replication với cluster 3 broker **13**
11. Xử lý lỗi thường gặp **16**
12. Dừng môi trường và nộp bằng chứng **17**
Tài liệu tham khảo **17**

1. Thiết kế buổi lab

1.1. Thời lượng và liên hệ với bài lý thuyết

Buổi lab dài **90 phút**. Phần 2–9 là phần bắt buộc, khoảng 65 phút. Phần 10 là phần mở rộng về replication, khoảng 20 phút; nếu lớp chỉ có 60 phút, có thể giao phần 10 làm bài về nhà.

Phần	Nội dung	Thời gian	Mục lý thuyết
2	Mô hình lab, khởi động Kafka	10 phút	3.1, 3.2
3	Tạo topic 3 partition	5 phút	4.1, 4.2
4–5	Gửi 60 record có key, đọc partition/offset	15 phút	4.2, 4.3, 5.1
6–7	Hai consumer cùng group, một group độc lập	12 phút	8.1, 8.2
8	Bốn consumer trên ba partition, rebalance	5 phút	8.1, 8.3
9	Consumer lag, reset offset và replay	13 phút	7.2, 7.4, 7.5
10	Mở rộng: replication với 3 broker	20 phút	6.1, 6.2, 6.3
11	Dọn môi trường, nộp bằng chứng	10 phút	Tổng hợp

1.2. Sản phẩm cần nộp

Mỗi học viên nộp một báo cáo ngắn (1–2 trang) gồm các bằng chứng **E1**–**E8** và câu trả lời cho các câu hỏi **Q1**–**Q8** ghi ở cuối từng phần. Bằng chứng là ảnh chụp terminal hoặc đoạn output dán vào báo cáo, kèm một câu giải thích.

Mã	Bằng chứng	Phần
E1	`--describe` của topic với đủ 3 partition	3
E2	Bảng đếm record theo partition, tổng 60	5
E3	Hai record của `o0001` kèm partition và offset	5
E4	`--members --verbose` của group `lab-stock` có hai client	6
E5	Group `lab-analytics-01` đọc đủ 62 record	7
E6	Assignment khi có bốn consumer, một consumer có 0 partition	8
E7	Bảng lag khi consumer dừng và bảng sau khi replay	9
E8 (mở rộng)	ISR trước/sau khi dừng `kafka-3` và lỗi `NOT_ENOUGH_REPLICAS`	10

1.3. Quy ước cửa sổ

Lab dùng một **terminal quản trị** và tối đa bốn cửa sổ consumer **A, B, C, D**. Mọi lệnh không ghi tên cửa sổ đều chạy ở terminal quản trị. Đặt terminal quản trị ở một thư mục làm việc riêng, ví dụ thư mục chứa bộ tài liệu này, vì bước 4 sẽ ghi file dữ liệu vào thư mục hiện tại.

2. Mô hình lab và khởi động Kafka

2.1. Mô hình lab một broker

Phần 2–9 dùng một container có tên `kafka-architecture-lab`. Topic `lab-order-events` có **3 partition**, nhưng **replication factor bằng 1** vì môi trường chỉ có một broker.

Thành phần	Giá trị dùng trong lab
Docker image	`apache/kafka:4.1.2`
Container	`kafka-architecture-lab`
Bootstrap server trong container	`localhost:9092`
Topic	`lab-order-events`
Partition	3
Replication factor	1
Key của record	`order_id`
Dữ liệu ban đầu	30 đơn, mỗi đơn có hai sự kiện, tổng cộng 60 record

Mô hình này đủ để quan sát partition, key, offset, group assignment và lag. Nó không kiểm chứng được replication giữa nhiều broker; phần 10 dùng một cluster 3 broker riêng cho nội dung đó.

2.2. Kiểm tra Docker và khởi động broker

Mở Docker Desktop, chờ engine sẵn sàng rồi mở PowerShell. Chạy:

PowerShell

```
docker version
```

Cần có thông tin **Server**, không chỉ Client. Nếu có lỗi kết nối Docker API hoặc named pipe, quay lại Docker Desktop trước khi chạy các bước tiếp theo.

Tải image và tạo container theo cách khởi động Docker trong Kafka Quick Start:

PowerShell

```
docker pull apache/kafka:4.1.2
docker run -d --name kafka-architecture-lab -p 127.0.0.1:9092:9092 apache/kafka
    :4.1.2
docker ps --filter name=kafka-architecture-lab
```

Nếu tên container đã được dùng từ lần trước, kiểm tra bằng `docker ps -a --filter name=kafka-architecture-lab`. Một lượt lab mới cần môi trường mới để giữ đúng số lượng record; xóa container cũ bằng `docker rm -f kafka-architecture-lab` trước khi chạy lại `docker run`.

Kiểm tra broker đã nhận yêu cầu được chưa:

PowerShell

```
docker exec kafka-architecture-lab /opt/kafka/bin/kafka-topics.sh --bootstrap-server
    localhost:9092 --list
```

Lệnh thành công nhưng chưa in topic nào là bình thường. Trong lượt chạy kiểm chứng, broker sẵn sàng sau khoảng 2 giây. Nếu lệnh báo lỗi kết nối, xem log và thử lại sau vài giây:

PowerShell

```
docker logs --tail 60 kafka-architecture-lab
```

Điểm kiểm tra: container đang chạy và lệnh liệt kê topic không còn báo lỗi kết nối broker.

3. Tạo topic với ba partition

PowerShell

```
docker exec kafka-architecture-lab /opt/kafka/bin/kafka-topics.sh --bootstrap-server
    localhost:9092 --create --topic lab-order-events --partitions 3 --replication-
    factor 1
docker exec kafka-architecture-lab /opt/kafka/bin/kafka-topics.sh --bootstrap-server
    localhost:9092 --describe --topic lab-order-events
```

Không dùng `--if-not-exists` ở đây để tránh vô tình tiếp tục trên topic đã chứa dữ liệu cũ. Xem các thao tác với topic.

Kết quả quan sát (lượt chạy 10/11/2026)

```
Created topic lab-order-events.
Topic: lab-order-events  TopicId: WoTwILKF...  PartitionCount: 3  ReplicationFactor:
    1
  Configs: min.insync.replicas=1,segment.bytes=1073741824
  Partition: 0  Leader: 1  Replicas: 1  Isr: 1  Elr:   LastKnownElr:
  Partition: 1  Leader: 1  Replicas: 1  Isr: 1  Elr:   LastKnownElr:
  Partition: 2  Leader: 1  Replicas: 1  Isr: 1  Elr:   LastKnownElr:
```

Đọc kết quả theo ba ý:

- `PartitionCount` là 3; có các partition 0, 1 và 2.
- `ReplicationFactor` là 1, nên mỗi partition chỉ có một replica trong `Replicas` và `Isr`.
- Broker `1` là leader của cả ba partition vì đây là broker duy nhất. Hai cột `Elr` và `LastKnownElr` liên quan đến cơ chế Eligible Leader Replicas, sẽ gặp lại ở phần 10.

E1: lưu output `--describe`.

Câu hỏi Q1. Topic này có bao nhiêu partition và bao nhiêu replica? Nếu tạo lại với replication factor 3 trên một cluster đủ broker, số replica là bao nhiêu?

4. Tạo và gửi 60 record có key

4.1. Tạo dữ liệu đơn hàng

Chạy đoạn PowerShell sau. Mỗi đơn có hai sự kiện theo thứ tự tạo đơn rồi thanh toán; mỗi dòng có cấu trúc `key|JSON`.

PowerShell

```
$events = foreach ($number in 1..30) {
    $orderId = "o{0:D4}" -f $number
    foreach ($eventType in @("OrderCreated", "OrderPaid")) {
        $value = [ordered]@{
            event_id = "$orderId-$eventType"
            event_type = $eventType
            order_id = $orderId
            amount = 450000
            currency = "VND"
        }
        '{0}|{1}' -f $orderId, ($value | ConvertTo-Json -Compress)
    }
}
$events.Count
$events | Select-Object -First 2
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
60
o0001|{"event_id":"o0001-OrderCreated","event_type":"OrderCreated","order_id":"o0001
    ","amount":450000,"currency":"VND"}
o0001|{"event_id":"o0001-OrderPaid","event_type":"OrderPaid","order_id":"o0001","
    amount":450000,"currency":"VND"}
```

4.2. Ghi file không BOM và gửi bằng console producer

Ghi dữ liệu ra file UTF-8 **không BOM**, chép file vào container rồi để console producer đọc file từ bên trong container:

PowerShell

```
$eventFile = Join-Path (Get-Location) 'lab-order-events.txt'
[System.IO.File]::WriteAllText($eventFile, ($events -join "`n") + "`n")
docker cp $eventFile kafka-architecture-lab:/tmp/lab-order-events.txt
docker exec kafka-architecture-lab sh -c "/opt/kafka/bin/kafka-console-producer.sh
    --bootstrap-server localhost:9092 --topic lab-order-events --property parse.key=
    true --property key.separator='|' --producer-property acks=all < /tmp/lab-order-
    events.txt"
```

Lệnh gửi thành công không in gì. `parse.key=true` yêu cầu console producer đọc phần trước dấu `|` làm key. Dấu phân cách chỉ phục vụ nhập liệu ở công cụ console; nó không phải cấu trúc bắt buộc của một Kafka record. Xem mã nguồn LineMessageReader.

Lưu ý

Vì sao không pipe thẳng `$events | docker exec -i ...`? Trong lượt chạy thử đầu tiên, cách pipe này trên Windows PowerShell 5.1 gắn ký tự BOM (`U+FEFF`) vào đầu dòng đầu tiên. Record đầu tiên vì thế có key `\uFEFFo0001`, khác với key `o0001` của record thứ hai. Hai key khác nhau có thể bị đưa vào hai partition khác nhau, làm hỏng chính phép quan sát "cùng key, cùng partition". Ghi file bằng `WriteAllText` rồi đọc trong container tránh được việc PowerShell mã hóa lại dữ liệu khi pipe.

Chỉ gửi file này **một lần**. Gửi lại sẽ tạo thêm 60 record; trùng `event_id` trong JSON không khiến Kafka bỏ qua record mới.

5. Đọc kèm partition, offset và key

Đọc lại toàn bộ dữ liệu bằng một group riêng, chỉ phục vụ việc quan sát, và tắt auto commit:

PowerShell

```
$lines = docker exec kafka-architecture-lab /opt/kafka/bin/kafka-console-consumer.sh
     --bootstrap-server localhost:9092 --topic lab-order-events --group lab-inspect
    -01 --from-beginning --max-messages 60 --timeout-ms 15000 --consumer-property
    enable.auto.commit=false --property print.partition=true --property print.offset
    =true --property print.key=true --property 'key.separator=|'
$lines | Select-Object -First 4
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
Processed a total of 60 messages
Partition:2|Offset:0|o0003|{"event_id":"o0003-OrderCreated",...}
Partition:2|Offset:1|o0003|{"event_id":"o0003-OrderPaid",...}
Partition:2|Offset:2|o0008|{"event_id":"o0008-OrderCreated",...}
Partition:2|Offset:3|o0008|{"event_id":"o0008-OrderPaid",...}
```

Dòng `Processed a total of 60 messages` được in ra luồng lỗi (stderr). Windows PowerShell 5.1 có thể hiển thị dòng này màu đỏ kèm chữ `NativeCommandError`; đây là thông báo bình thường, không phải lỗi. Xem DefaultMessageFormatter về các thuộc tính `print.*`.

Đếm record theo partition:

PowerShell

```
$partitionIds = $lines | ForEach-Object { if ($_ -match 'Partition:(\d+)') {
    $Matches[1] } }
$partitionIds.Count
$partitionIds | Group-Object | Sort-Object Name | Select-Object Name, Count
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
60

Name Count
---- -----
0       20
1       22
2       18
```

Phân phối không đều (20/22/18) là bình thường: partitioner băm key thành số partition, không chia lần lượt. Với cùng 30 key, cùng số partition và cùng client Java mặc định, phép băm cho ra cùng kết quả, nên bạn có thể thấy đúng các con số này. Nếu số khác, hãy kiểm tra số lần gửi và key có bị BOM hay không trước khi kết luận.

Kiểm tra hai sự kiện của đơn đầu tiên:

PowerShell

```
$lines | Where-Object { $_ -match '\|o0001\|' }
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
Partition:0|Offset:0|o0001|{"event_id":"o0001-OrderCreated",...}
Partition:0|Offset:1|o0001|{"event_id":"o0001-OrderPaid",...}
```

Hai record cùng key nằm ở cùng partition và `OrderCreated` có offset nhỏ hơn `OrderPaid`. Thứ tự in ra giữa các partition không phải thứ tự nghiệp vụ toàn cục: lượt đọc có thể in P2 trước P0.

E2, E3: lưu bảng đếm và hai dòng của `o0001`.

Câu hỏi Q2. Offset 0 xuất hiện ở cả P0, P1 và P2. Vì sao đây không phải là ba bản sao của cùng một record?

Câu hỏi Q3. Nếu topic được tạo với 4 partition thay vì 3, bạn có chắc `o0001` vẫn vào P0 không? Điều gì vẫn được giữ?

6. Hai consumer chia việc trong cùng group

Mở **hai cửa sổ PowerShell mới** và giữ các consumer chạy trong suốt phần 6–8.

Cửa sổ A:

PowerShell

```
docker exec -it kafka-architecture-lab /opt/kafka/bin/kafka-console-consumer.sh --
    bootstrap-server localhost:9092 --topic lab-order-events --group lab-stock --
    from-beginning --consumer-property client.id=stock-c1 --consumer-property enable
    .auto.commit=true --consumer-property auto.commit.interval.ms=1000 --property
    print.partition=true --property print.offset=true --property print.key=true
```

Cửa sổ B (mở sau khi cửa sổ A đã in dữ liệu):

PowerShell

```
docker exec -it kafka-architecture-lab /opt/kafka/bin/kafka-console-consumer.sh --
    bootstrap-server localhost:9092 --topic lab-order-events --group lab-stock --
    from-beginning --consumer-property client.id=stock-c2 --consumer-property enable
    .auto.commit=true --consumer-property auto.commit.interval.ms=1000 --property
    print.partition=true --property print.offset=true --property print.key=true
```

Khoảng 10 giây sau, kiểm tra thành viên và partition được giao từ terminal quản trị:

PowerShell

```
docker exec kafka-architecture-lab /opt/kafka/bin/kafka-consumer-groups.sh --
    bootstrap-server localhost:9092 --describe --group lab-stock --members --verbose
docker exec kafka-architecture-lab /opt/kafka/bin/kafka-consumer-groups.sh --
    bootstrap-server localhost:9092 --describe --group lab-stock
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
GROUP      CONSUMER-ID          CLIENT-ID  #PARTITIONS  CURRENT-ASSIGNMENT
lab-stock  stock-c1-95894cec... stock-c1   2            lab-order-events:0,1
lab-stock  stock-c2-9be19f4d... stock-c2   1            lab-order-events:2

GROUP      TOPIC             PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG  CLIENT-
    ID
lab-stock  lab-order-events  0          20              20              0    stock-
    c1
lab-stock  lab-order-events  1          22              22              0    stock-
    c1
lab-stock  lab-order-events  2          18              18              0    stock-
    c2
```

Trong lượt chạy kiểm chứng, cửa sổ A in đủ **60 record**, còn cửa sổ B **không in record nào**, dù `stock-c2` đang sở hữu P2. Lý do: A khởi động trước, nhận cả ba partition, đọc hết và commit offset. Khi B tham gia, group rebalance và giao P2 cho B; B tiếp tục từ committed offset 18 của P2 nên không còn gì để đọc. `--from-beginning` chỉ có tác dụng khi group chưa có committed offset.

Gửi thêm hai sự kiện mới để thấy B làm việc:

PowerShell

```
$newFile = Join-Path (Get-Location) 'lab-new-events.txt'
[System.IO.File]::WriteAllText($newFile, (@('o9991|{"event_id":"e-new-1","event_type":"OrderCreated","order_id":"o9991"}', 'o9992|{"event_id":"e-new-2","event_type":"OrderCreated","order_id":"o9992"}') -join "`n") + "`n")
docker cp $newFile kafka-architecture-lab:/tmp/lab-new-events.txt
docker exec kafka-architecture-lab sh -c "/opt/kafka/bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic lab-order-events --property parse.key=true --property key.separator='|' < /tmp/lab-new-events.txt"
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
(A) Partition:0  Offset:20  o9992  {"event_id":"e-new-2",...}
(B) Partition:2  Offset:18  o9991  {"event_id":"e-new-1",...}
```

Dòng `(A)` là cửa sổ A, dòng `(B)` là cửa sổ B. Record `o9991` vào P2 nên xuất hiện ở B; `o9992` vào P0 nên xuất hiện ở A. Chạy lại `--describe` sẽ thấy `LOG-END-OFFSET` của P0 và P2 tăng thêm 1 và `LAG` về 0 sau chu kỳ auto commit. Auto commit trong ví dụ console chỉ phản ánh tiến độ đọc; nó không chứng minh một thao tác nghiệp vụ như trừ tồn kho đã hoàn tất. Xem quản lý consumer group.

E4: lưu output `--members --verbose`.

Câu hỏi Q4. Cửa sổ B không in record nào trước khi gửi dữ liệu mới. Từ bằng chứng nào bạn biết B vẫn được phân công partition?

7. Đọc độc lập bằng một group khác

Giữ A và B chạy. Từ terminal quản trị, dùng một group mới để đọc toàn bộ record đang được lưu:

PowerShell

```
docker exec kafka-architecture-lab /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic lab-order-events --group lab-analytics-01 --from-beginning --max-messages 62 --timeout-ms 15000 --consumer-property enable.auto.commit=false --property print.partition=true --property print.offset=true --property print.key=true
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
...
Processed a total of 62 messages
```

Group mới đọc đủ **62 record** (60 ban đầu và 2 record mới), dù group `lab-stock` đã đọc và commit các record đó. Commit của một group không làm mất dữ liệu của topic và không ảnh hưởng tiến độ của group khác. Xem consumer offset reset.

E5: lưu dòng `Processed a total of 62 messages`.

8. Bốn consumer trên ba partition

Mở thêm cửa sổ **C** và **D** với cùng `group.id=lab-stock`, chỉ đổi `client.id`:

PowerShell

```
docker exec -it kafka-architecture-lab /opt/kafka/bin/kafka-console-consumer.sh --
    bootstrap-server localhost:9092 --topic lab-order-events --group lab-stock --
    consumer-property client.id=stock-c3 --property print.partition=true --property
    print.offset=true --property print.key=true
```

Cửa sổ D chạy cùng lệnh với `client.id=stock-c4`. Chờ khoảng 10 giây rồi kiểm tra:

PowerShell

```
docker exec kafka-architecture-lab /opt/kafka/bin/kafka-consumer-groups.sh --
    bootstrap-server localhost:9092 --describe --group lab-stock --members --verbose
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
GROUP      CONSUMER-ID          CLIENT-ID  #PARTITIONS  CURRENT-ASSIGNMENT
lab-stock  stock-c1-95894cec... stock-c1   1            lab-order-events:0
lab-stock  stock-c3-2a5fccc8... stock-c3   1            lab-order-events:2
lab-stock  stock-c4-5f11ed59... stock-c4   0            -
lab-stock  stock-c2-9be19f4d... stock-c2   1            lab-order-events:1
```

Sau rebalance, mỗi partition thuộc đúng một consumer và **một consumer có 0 partition**. Lưu ý `stock-c2` đã chuyển từ P2 sang P1: không nên giả định một consumer luôn giữ một partition cố định. Consumer cụ thể nào bị để trống có thể khác trên máy bạn.

E6: lưu output có dòng `#PARTITIONS` bằng 0.

Câu hỏi Q5. Thêm consumer thứ năm vào `lab-stock` có làm group đọc nhanh hơn không? Muốn có 5 consumer cùng làm việc thì phải thay đổi gì, và thay đổi đó ảnh hưởng gì đến ánh xạ key?

9. Consumer lag, reset offset và replay

9.1. Tạo lag

Dừng cả bốn cửa sổ A–D bằng `Ctrl+C`. Group `lab-stock` giờ không còn thành viên nhưng committed offset vẫn được lưu. Gửi thêm 10 đơn mới:

PowerShell

```
$more = foreach ($number in 31..40) { $o = "o{0:D4}" -f $number; '{0}|{{"event_id
    ":"{0}-OrderCreated","event_type":"OrderCreated","order_id":"{0}"}}' -f $o }
$moreFile = Join-Path (Get-Location) 'lab-more-events.txt'
[System.IO.File]::WriteAllText($moreFile, ($more -join "`n") + "`n")
docker cp $moreFile kafka-architecture-lab:/tmp/lab-more-events.txt
docker exec kafka-architecture-lab sh -c "/opt/kafka/bin/kafka-console-producer.sh
    --bootstrap-server localhost:9092 --topic lab-order-events --property parse.key=
    true --property key.separator='|' < /tmp/lab-more-events.txt"
docker exec kafka-architecture-lab /opt/kafka/bin/kafka-consumer-groups.sh --
    bootstrap-server localhost:9092 --describe --group lab-stock
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
Consumer group 'lab-stock' has no active members.

GROUP      TOPIC             PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
    CONSUMER-ID
lab-stock  lab-order-events  0          21              22              1    -
lab-stock  lab-order-events  1          22              25              3    -
lab-stock  lab-order-events  2          19              25              6    -
```

Tổng lag là $1 + 3 + 6 = \mathbf{10}$, đúng bằng số record mới. Lag là khoảng cách offset giữa committed offset của group và cuối log của từng partition. Nó không đo bằng giây, và với dữ liệu thực tế cũng không luôn bằng số công việc nghiệp vụ còn lại.

9.2. Đặt lại offset và đọc lại

Đặt lại committed offset của `lab-stock` về đầu log. Luôn chạy `--dry-run` trước để xem kết quả dự kiến, rồi mới `--execute`. Lệnh chỉ chạy được khi group không có thành viên hoạt động.

PowerShell

```
docker exec kafka-architecture-lab /opt/kafka/bin/kafka-consumer-groups.sh --
    bootstrap-server localhost:9092 --group lab-stock --topic lab-order-events --
    reset-offsets --to-earliest --dry-run
docker exec kafka-architecture-lab /opt/kafka/bin/kafka-consumer-groups.sh --
    bootstrap-server localhost:9092 --group lab-stock --topic lab-order-events --
    reset-offsets --to-earliest --execute
docker exec kafka-architecture-lab /opt/kafka/bin/kafka-consumer-groups.sh --
    bootstrap-server localhost:9092 --describe --group lab-stock
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
GROUP      TOPIC              PARTITION  NEW-OFFSET
lab-stock  lab-order-events   0          0
lab-stock  lab-order-events   1          0
lab-stock  lab-order-events   2          0

GROUP      TOPIC              PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
lab-stock  lab-order-events   0          0               22              22
lab-stock  lab-order-events   1          0               25              25
lab-stock  lab-order-events   2          0               25              25
```

Chạy lại một consumer của `lab-stock` để replay toàn bộ log:

PowerShell

```
1 docker exec kafka-architecture-lab /opt/kafka/bin/kafka-console-consumer.sh --
      bootstrap-server localhost:9092 --topic lab-order-events --group lab-stock --max
      -messages 72 --timeout-ms 15000 --property print.partition=true --property print
      .offset=true
2 docker exec kafka-architecture-lab /opt/kafka/bin/kafka-consumer-groups.sh --
      bootstrap-server localhost:9092 --describe --group lab-stock
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
...
Processed a total of 72 messages

GROUP      TOPIC              PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
lab-stock  lab-order-events   0          22              22              0
lab-stock  lab-order-events   1          25              25              0
lab-stock  lab-order-events   2          25              25              0
```

Group đọc lại đủ **72 record** (60 + 2 + 10) vì dữ liệu vẫn nằm trong retention của topic. Lệnh này không có `--from-beginning`: vị trí bắt đầu đến từ committed offset vừa được đặt lại.

E7: lưu bảng lag ở 9.1 và bảng sau replay.

Câu hỏi Q6. Nếu ứng dụng tồn kho thật đã trừ kho cho 62 sự kiện đầu, việc replay ở trên gây hậu quả gì? Ứng dụng cần thiết kế gì để replay an toàn?

10. Mở rộng: replication với cluster 3 broker

10.1. Khởi động cluster

Phần này dùng file docker-compose.replication.yml: ba node `kafka-1`, `kafka-2`, `kafka-3`, mỗi node vừa là broker vừa là controller (KRaft). Cluster không mở cổng ra máy host; mọi lệnh chạy bằng `docker exec kafka-1`. Có thể giữ hoặc dừng container của phần 2–9, hai môi trường không dùng chung cổng.

Từ thư mục `slaivn`, chạy:

PowerShell

```
docker compose -f kafka/lab/docker-compose.replication.yml up -d
docker exec kafka-1 /opt/kafka/bin/kafka-metadata-quorum.sh --bootstrap-server kafka
    -1:9092 describe --status
```

Nếu lệnh thứ hai báo lỗi kết nối, chờ vài giây rồi chạy lại. Kết quả cần có `CurrentVoters` với ba node 1, 2 và 3.

10.2. Topic replication factor 3

PowerShell

```
docker exec kafka-1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka-1:9092
    --create --topic lab-replicated --partitions 3 --replication-factor 3 --config
    min.insync.replicas=2
docker exec kafka-1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka-1:9092
    --describe --topic lab-replicated
docker exec kafka-1 sh -c "printf 'o0001|created\no0002|created\no0003|created\n' |
    /opt/kafka/bin/kafka-console-producer.sh --bootstrap-server kafka-1:9092 --topic
     lab-replicated --property parse.key=true --property key.separator='|' --
    producer-property acks=all"
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
Partition: 0  Leader: 1  Replicas: 1,2,3  Isr: 1,2,3
Partition: 1  Leader: 2  Replicas: 2,3,1  Isr: 2,3,1
Partition: 2  Leader: 3  Replicas: 3,1,2  Isr: 3,1,2
```

Có 3 partition logic và 9 replica. Leader được phân bố trên ba broker; mỗi broker vừa là leader của một partition vừa giữ follower của hai partition còn lại, đúng như Hình 4 của bài lý thuyết. Lệnh gửi `acks=all` thành công không in gì.

10.3. Mất một broker

PowerShell

```
docker stop kafka-3
docker exec kafka-1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka-1:9092
    --describe --topic lab-replicated
docker exec kafka-1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka-1:9092
    --describe --topic lab-replicated --under-replicated-partitions
docker exec kafka-1 sh -c "printf 'o0001|paid\no0002|paid\no0003|paid\n' | /opt/
    kafka/bin/kafka-console-producer.sh --bootstrap-server kafka-1:9092 --topic lab-
    replicated --property parse.key=true --property key.separator='|' --producer-
    property acks=all"
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
Partition: 0  Leader: 1  Replicas: 1,2,3  Isr: 1,2
Partition: 1  Leader: 2  Replicas: 2,3,1  Isr: 2,1
Partition: 2  Leader: 1  Replicas: 3,1,2  Isr: 1,2
```

Broker 3 bị loại khỏi ISR của cả ba partition. P2 mất leader nên được bầu leader mới là broker 1. `Replicas` không đổi: đó là danh sách replica được gán, không phải danh sách đang theo kịp. Cả ba partition xuất hiện trong `--under-replicated-partitions`. ISR còn 2, bằng `min.insync.replicas`, nên lần gửi `acks=all` vẫn thành công.

10.4. ISR thấp hơn ngưỡng tối thiểu

Nâng ngưỡng lên 3 trong khi `kafka-3` vẫn dừng, rồi gửi bằng `acks=all`:

PowerShell

```
docker exec kafka-1 /opt/kafka/bin/kafka-configs.sh --bootstrap-server kafka-1:9092
    --alter --entity-type topics --entity-name lab-replicated --add-config min.
    insync.replicas=3
docker exec kafka-1 sh -c "printf 'o0004|created\n' | /opt/kafka/bin/kafka-console-
    producer.sh --bootstrap-server kafka-1:9092 --topic lab-replicated --property
    parse.key=true --property key.separator='|' --producer-property acks=all --
    producer-property delivery.timeout.ms=10000 --producer-property request.timeout.
    ms=5000"
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
WARN ... on topic-partition lab-replicated-0, retrying (2 attempts left). Error:
    NOT_ENOUGH_REPLICAS
WARN ... retrying (1 attempts left). Error: NOT_ENOUGH_REPLICAS
WARN ... retrying (0 attempts left). Error: NOT_ENOUGH_REPLICAS
ERROR Error when sending message to topic lab-replicated with key: 5 bytes, value: 7
     bytes ...
```

Leader từ chối ghi vì ISR (2) nhỏ hơn `min.insync.replicas` (3). Chú ý: console producer vẫn kết thúc với exit code 0, nên phải đọc dòng `ERROR` thay vì chỉ dựa vào việc lệnh kết thúc.

Thử cùng record với `acks=1`, rồi xem offset cuối của từng partition:

PowerShell

```
docker exec kafka-1 sh -c "printf 'o0004|created\n' | /opt/kafka/bin/kafka-console-producer.sh --bootstrap-server kafka-1:9092 --topic lab-replicated --property parse.key=true --property key.separator='|' --producer-property acks=1"
docker exec kafka-1 /opt/kafka/bin/kafka-get-offsets.sh --bootstrap-server kafka-1:9092 --topic lab-replicated
docker exec kafka-1 /opt/kafka/bin/kafka-features.sh --bootstrap-server kafka-1:9092 describe
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
lab-replicated:0:4
lab-replicated:1:0
lab-replicated:2:2
Feature: eligible.leader.replicas.version  SupportedMinVersion: 0  SupportedMaxVersion: 1  FinalizedVersionLevel: 1
```

Lần gửi `acks=1` không báo lỗi vì producer chỉ chờ leader ghi vào log cục bộ. Nhưng offset cuối của P0 vẫn là **4**: record mới chưa đọc được. Cluster Kafka 4.1.2 mới tạo có **ELR bật mặc định** (`eligible.leader.replicas.version` ở mức 1). Khi ELR bật, high watermark của partition chỉ tăng khi ISR đạt ít nhất `min.insync.replicas`, nên consumer chưa thấy record. Đây chính là thay đổi ngữ nghĩa mà ghi chú ELR ở mục 6.3 bài lý thuyết nhắc tới; ví dụ trong bài đọc mô tả trường hợp ELR tắt. Xem ghi chú ELR trong cấu hình topic.

10.5. Broker quay lại

PowerShell

```
docker start kafka-3
docker exec kafka-1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka-1:9092 --describe --topic lab-replicated
docker exec kafka-1 /opt/kafka/bin/kafka-get-offsets.sh --bootstrap-server kafka-1:9092 --topic lab-replicated
```

Chờ khoảng 20 giây nếu ISR chưa đủ ba replica rồi chạy lại.

Kết quả quan sát (lượt chạy 10/11/2026)

```
Partition: 0  Leader: 1  Replicas: 1,2,3  Isr: 1,2,3
Partition: 1  Leader: 2  Replicas: 2,3,1  Isr: 2,1,3
Partition: 2  Leader: 1  Replicas: 3,1,2  Isr: 1,2,3
lab-replicated:0:5
lab-replicated:1:0
lab-replicated:2:2
```

Broker 3 bắt kịp và trở lại ISR. ISR đạt ngưỡng 3 nên high watermark của P0 tăng lên 5: record `o0004` gửi bằng `acks=1` giờ mới đọc được. Leader của P2 vẫn là broker 1. Kafka tự trả leader về replica ưu tiên sau một chu kỳ kiểm tra; có thể làm ngay bằng lệnh:

PowerShell

```
docker exec kafka-1 /opt/kafka/bin/kafka-leader-election.sh --bootstrap-server kafka
    -1:9092 --election-type preferred --topic lab-replicated --partition 2
```

Kết quả quan sát (lượt chạy 10/11/2026)

```
Successfully completed leader election (PREFERRED) for partitions lab-replicated-2
```

E8: lưu ISR ở 10.2 và 10.3 cùng dòng `NOT_ENOUGH_REPLICAS` ở 10.4.

Câu hỏi Q7. Ở 10.3, broker 3 đã dừng nhưng vẫn có trong `Replicas`. Hai cột `Replicas` và `Isr` khác nhau ở ý nghĩa nào?

Câu hỏi Q8. Ở 10.4, producer `acks=1` nhận xác nhận nhưng consumer chưa đọc được record. Điều đó nói gì về câu "producer nhận ack nghĩa là dữ liệu đã an toàn và sẵn sàng"?

Dọn cluster khi xong:

PowerShell

```
docker compose -f kafka/lab/docker-compose.replication.yml down -v
```

11. Xử lý lỗi thường gặp

Hiện tượng	Cách kiểm tra
Không kết nối Docker API	Mở Docker Desktop, chờ engine chạy và kiểm tra lại `docker version`
Container thoát hoặc broker chưa sẵn sàng	Xem `docker ps -a` và `docker logs --tail 60 kafka-architecture-lab`
Cổng 9092 đang được dùng	Tìm tiến trình hoặc container đang giữ cổng; nếu chỉ đổi host port, các lệnh `docker exec` vẫn dùng cổng 9092 bên trong container
Topic đã tồn tại	Xóa container và chạy lại từ phần 2; không dùng dữ liệu của lượt trước
Hai record của `o0001` ở hai partition khác nhau	Key đầu tiên có thể bị gắn BOM; dùng cách ghi file ở 4.2 và gửi lại trên môi trường mới
Dòng `Processed a total of ...` màu đỏ kèm `NativeCommandError`	Thông báo in ra stderr; Windows PowerShell 5.1 hiển thị như lỗi nhưng lệnh đã chạy đúng
Consumer chưa đủ 60 record rồi timeout	Kiểm tra số lần gửi, tên topic và offset; tăng `--timeout-ms` nếu máy chậm
Cửa sổ consumer mới không in gì	Group đã có committed offset; kiểm tra assignment rồi gửi thêm dữ liệu

Hiện tượng	Cách kiểm tra
`--reset-offsets` báo group đang hoạt động	Dừng mọi consumer của group bằng `Ctrl+C`, chờ vài giây rồi chạy lại
Producer `acks=all` báo `NOT_ENOUGH_REPLICAS`	ISR nhỏ hơn `min.insync.replicas`; đây là kết quả mong đợi ở 10.4
Cảnh báo `Couldn't resolve server kafka-3`	Có trong danh sách bootstrap một broker đã dừng; dùng `kafka-1:9092` như trong lab

12. Dừng môi trường và nộp bằng chứng

Dừng mọi consumer còn chạy bằng `Ctrl+C`. Nếu muốn giữ dữ liệu để học tiếp, chỉ dừng container:

PowerShell

```
docker stop kafka-architecture-lab
```

Nếu đã hoàn tất, xóa container lab cùng dữ liệu bên trong:

PowerShell

```
docker rm -f kafka-architecture-lab
```

Trước khi dọn, kiểm tra báo cáo đã có đủ E1–E7 (và E8 nếu làm phần mở rộng), câu trả lời Q1–Q8, phiên bản image và số record thực tế đã gửi.

Tài liệu tham khảo

- Kafka Quick Start: khởi động Docker, ghi và đọc sự kiện.
- Basic Kafka Operations: topic, consumer group và reset offset.
- LineMessageReader của Kafka 4.1.2: nhập key/value ở console producer.
- DefaultMessageFormatter của Kafka 4.1.2: in partition, offset và key.
- Consumer Configs: auto commit và offset reset.
- Topic Configs: `min.insync.replicas` và ghi chú ELR.
- Docker image apache/kafka: cấu hình bằng biến môi trường.